

Visual Programming Tools for Young Learners: Investigating How Different Programming Paradigms Teach Software Architecture Concepts

1st Oliver Wagner
owagn20@student.sdu.dk

2nd Marcus Elmkvist
maelm22@student.sdu.dk

3rd Toke Odgaard-Jans
toodg22@student.sdu.dk

4th Henrik Prüß
hepru23@student.sdu.dk

Abstract—Xxxxx

I. INTRODUCTION

This is the introduction chapter.

II. RELATED WORK

This is the related work chapter.

A. *Software Engineering in Education*

When people think of software engineering, they often focus on programming, but software engineering education prepares students not only to write code but also to design, develop, and maintain complex software systems.

1) *Programming Education*: Though programming is not the main focus of software engineering it serves as the entry point to software engineering, where students learn fundamental concepts that later extend into more advanced topics such as software design and system architecture.

Because programming serves as the entry point to software engineering, the choice of programming language is important, as languages with a steep learning curve may discourage beginners from engaging with the field.

The specific language to use for this entry point has been a history of debate [1], as language preference is subject to personal preference and efficiency varies between learners. George Milbrandt [2] suggested these criteria for language selection for beginners: it should be easy to use, have a structured design with one-entry/one-exit flow, offer powerful computing capacity while maintaining simple syntax, support variable declaration and expressive, meaningful variable names, provide easy input/output and output formatting, give immediate feedback, and include good diagnostic tools for testing and debugging. Although these are good criteria for a language to have its leaves very little tooling for comparing different languages and their capacity to teach. This is something that Mannila and Raadt have sought to rectify in their paper "*An Objective Comparison of Languages for Teaching Introductory Programming*" [2], wherein they compare different programming language to find the one which is the best for teaching beginners. They have collected 17 different criteria drawn from the design decisions made by creators of languages that are considered "teaching languages".

2) *Challenges in Learning Software Engineering*:

3) *Computational Thinking and Conceptual Learning*:

4) *Game-Based Learning*:

III. RESEARCH METHOD

IV. RESEARCH METHOD

This is the research method chapter.

V. EXPERIMENT

This is the experiment chapter.

VI. RESULTS

VII. RESULTS

This chapter presents the results of the experiment and associated data collection methods mentioned in ??.

A. *Survey*

1) *Pre-Survey*: The pre-survey focused on two aspects: it should capture the students' previous programming experience, and it should gauge their confidence in explaining four selected software design concepts.

All students reported previous programming experience. Confidence in being able to solve complex programming tasks was low. However, all students indicated that they had some experience in using a visual programming language (Scratch). Further, both the Scratch student and one of the Godot Orchestrator students said that they had programmed in a textual programming language before too. Regarding the to-be-analyzed programming concepts (modularity, instantiation, state management, event handling), confidence in explaining any of them was rather low, with the exception of instantiation.

The Scratch student indicated both the most experience with programming and the highest confidence in explaining the different programming concepts: instantiation and event handling are rated higher than modularity and state management, however the student is still confident in explaining the latter concepts too. [TODO: COMPARISON WITH GODOT? Are even his lower ratings than the Godot ratings?]

The two Godot Orchestrator students varied in their experience. One student reported some programming experience, mostly visual-based programming. The other student seemed to have considerable programming experience, even with creating games. The latter had used more textual programming

languages than visual programming languages in the past. Overall, both Godot Orchestrator students indicated low confidence in explaining any of the four programming principles.

2) *Post-Survey*: The post-survey focused on two aspects: it should capture the students' experience with their respective visual programming language during the workshop, and it should again gauge their confidence in explaining the same four selected software design concepts as in the pre-survey.

Independently of which VPL they used, students reported high confidence in building large programs with their respective VPL. Confidence in solving complex problems with VPLs was reported low similar to the pre-survey results. Taken together, students using either of the two languages reported that their language might help with addressing modularity in their code, however they saw problems for state management. Results for event handling and instantiation were mixed. Contrary, confidence in explaining was highest in event handling and state management, with mixed results for modularity and instantiation.

The Scratch student evaluated Scratch's ability to achieve modularity, instantiation and event handling as high, whereas state management seemed harder in Scratch. In addition, the Scratch student indicated increased confidence in explaining in any of the four concepts, compared to the results of the pre-survey.

The other students saw Godot Orchestrator's ability more critical. One student did not find the language useful for implementing any of the four concepts. The other student found it at least useful for modularity and instantiation. Regarding their confidence in explaining the four concepts it was lower than what the Scratch user reported. One Godot Orchestrator reported low confidence in explaining any of the concepts. The other student answers with high confidence in explaining event handling. State management was rated neutral from both Godot Orchestrator students.

Fig. 1 shows pre-survey answers mapped to their post-survey equivalents. It shows how confident students were explaining in each of the four concepts. All data points left of the diagonal line show an improvement, whereas all data points on the right indicate that confidence went down. Of the four concepts only instantiation stagnated or even decreased, whereas the others mostly improved for both Scratch and Godot users. Overall, the understanding of state management and event handling seem to have improved most, according to the surveys.

As Fig. 2 suggests, both visual programming languages performed differently in conveying the four concepts. Based on the collected survey answers, Scratch seems to have better explained state management, whereas Godot Orchestrator positively influenced the students' confidence in explaining event handling. Modularity worked well in both languages, and instantiation appears hard to grasp in Godot Orchestrator. Scratch on the other hand seemed to have no effect on the students' confidence in explaining instantiation, which was already high in the pre-survey. Overall, both languages seem

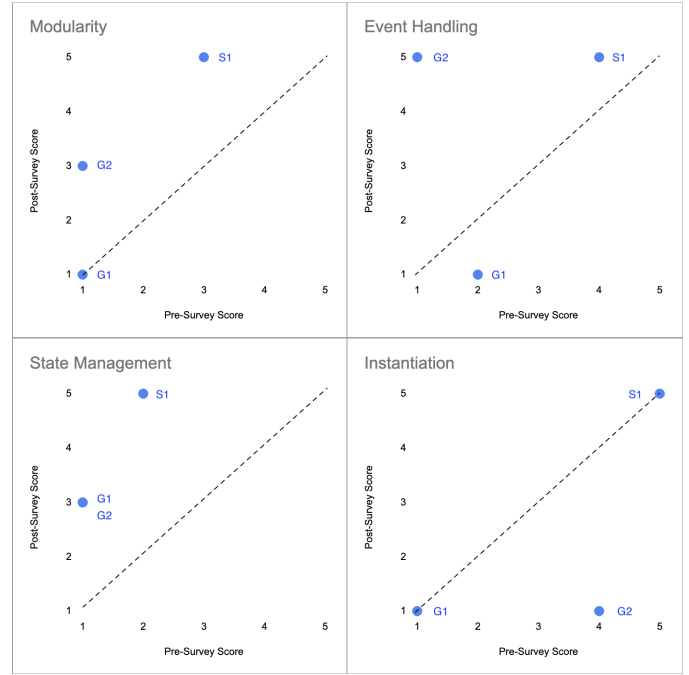


Fig. 1. Pre- and post-survey answers for each participant (S1: Scratch, G1+G2: Godot)

to have improved the students' confidence in explaining the software design concepts.

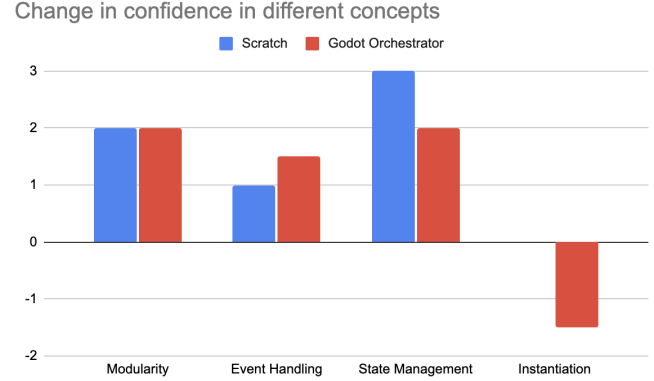


Fig. 2. Change in confidence in different concepts

B. Multiple Choice Test

Due to an organizational mistake, we only conducted the multiple choice test after the workshop, instead of both before and after. The idea was to run the same test two times to achieve comparability. This is described in more detail in IV. Acknowledging our mistake, we still want to present the results of the post test.

Overall, the Scratch student scored considerably higher than both Godot Orchestrator students. While the Scratch student answered 50% of the questions correctly, the Godot Orchestrator students scored 20% and 0% respectively. The

latter succeeded only with state management-related questions, whereas the former had correct answers related to modularity, event handling and state management. None of the students answered any question concerning instantiation correctly. Across all students modularity and state management were more successful than instantiation and event handling.

As we did not conduct the pre multiple choice test there is no direct comparison possible. However, we can relate the test results to the survey results mentioned before. Instantiation was reported to be the most difficult concept during pre-survey, which aligns with the low test results. The Scratch student reported high confidence in explaining instantiation, however they still scored 0 points with instantiation-related questions.

Modularity and state management were initially rated difficult too, however the post survey showed an increase in confidence. The Scratch student indicated a better understanding of those two concepts, and scored on related questions. One of the Godot Orchestrator students did not feel improvement, which was accordingly reflected in their test results (i.e., they scored 0%). The other student reported a smaller improvement than the Scratch student, however they only answered state management-related questions correctly.

Event handling seems overall to have been a hard to grasp concept in our workshop. The Scratch student answered answered one of the two related questions correctly, whereas both Godot Orchestrator students struggled with both questions. Those results are contrary to the post survey results saying that two of the students had high confidence in understanding the concept. It also somewhat contradicts the finding that the confidence in explaining event handling increased the second most from pre to post survey. It however aligns mostly with the pre survey evaluation of event handling, which described it as a difficult topic.

C. Workshop Observation

While conducting the programming workshop, including lecture and project work, we took notes of the students' interaction with their respective visual programming language.

VIII. DISCUSSION

This is the discussion chapter.

IX. CONCLUSION

This is the conclusion chapter.

REFERENCES

- [1] Villagómez-Palacios, C. De la Fuente-Burdiles, and C. Vidal-Silva, "Bridging the Gap Between Text-Based and Visual Programming: A Comparative Study of Efficiency and Student Engagement in Game Development," *International Journal of Advanced Computer Science and Applications*, vol. 16, no. 12, pp. 1249 – 1256, 2025.
- [2] L. Mannila and M. de Raadt, "An objective comparison of languages for teaching introductory programming," in *Proceedings of the 6th Baltic Sea conference on Computing education research: Koli Calling 2006*, Baltic Sea '06, (New York, NY, USA), pp. 32–37, Association for Computing Machinery, Feb. 2006.